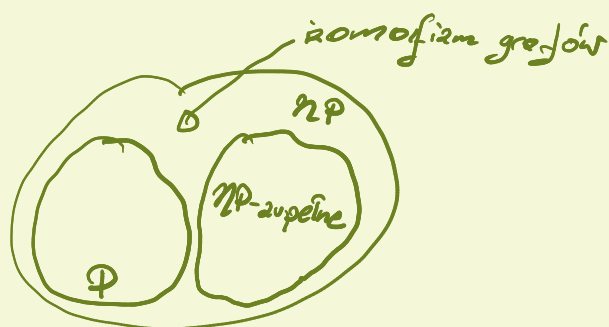




Wybór k -tego elementu

Dane: $a_1, \dots, a_n$
 $k \in \mathbb{N}$

Wynik: a_i , t.j. $|\{a_j < a_i < a_k\}| = k-1$
 czyli k -te minimum.

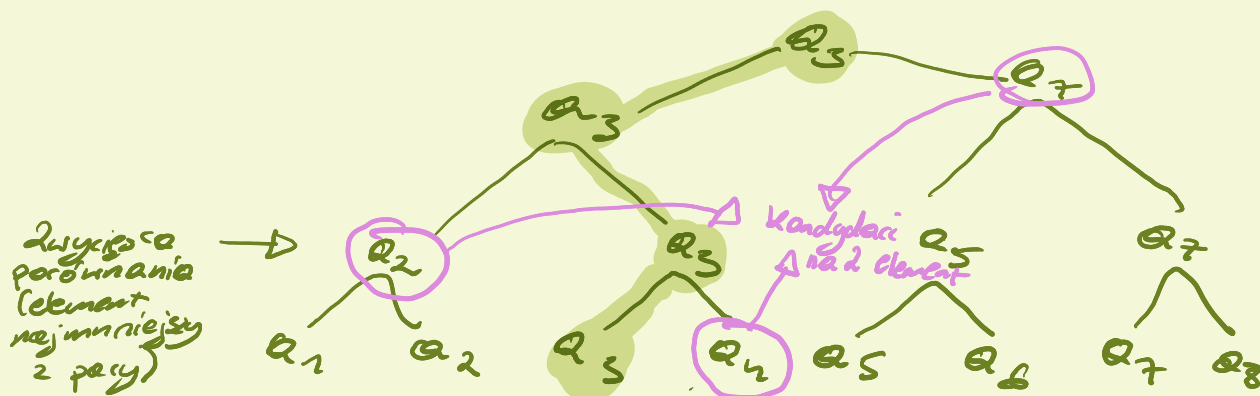
Model - drzewa decyzyjne

$k=1$

potrzeba i wystarczy $n-1$ porównań

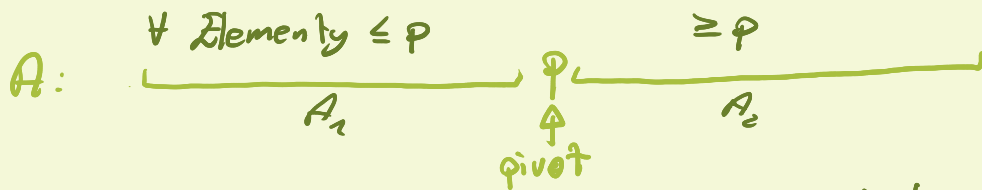
$k=2$

najmniej: $2n-3$ porównania (tymczasowe zmienne $\text{min1st}, \text{min2nd}$ i porównujemy każdy element z tymi dwoma).



Przypadek ogólny - wykorzystujemy partition.

Do przeprowadzenia partition:



Jeżeli p jest na pozycji i -tej to jest on i -tym elementem

Jeżeli $i = k$ to problem z głowy.

Jeżeli $i < k$ (znaleziliśmy za mały element) k -tego elementu
szukamy w części A_2 .

Select($A[b \dots e], k$)

1) $p \leftarrow$ pivot z $A[b \dots e]$

$i \leftarrow$ partition($A[b \dots e], p$)

if $i = k$ return p

if $i < k$ return Select($A[p+1, \dots, e], k$)

return select($A[b, \dots, p-1], k$)

Problem (taki sam jak z Quick Sortem) - jak dobrać p ?

Zrobimy to deterministycznie.

Algorytm magicznych piątek

- W zasadzie to NIE jest osobny algorytm - tylko pewna ilość kroków wewnątrz select + pewne dodatkowe ugięcie rekurencyjne selecta. Robimy coś takiego:

1° Dzielimy A na piątki (1 niepełna)

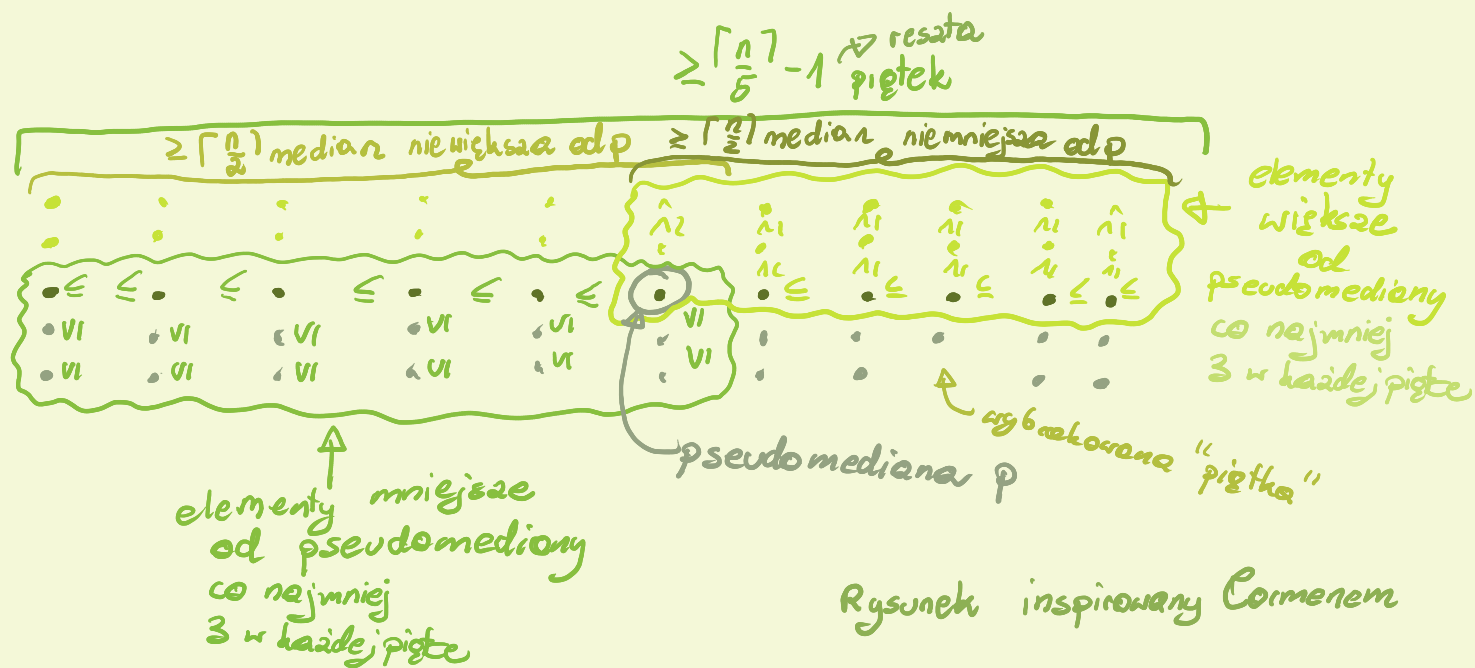
2° Wyznaczamy S - zbiór median tych piątek $O(1)$

3° Wywołujemy $p \leftarrow$ select($S, \lceil \frac{|S|}{2} \rceil$) i dostajemy medianę S (dalej pseudomediana), bo $\lceil \frac{|S|}{2} \rceil$ -ty element to mediana S .

Wtedy p będzie naszym pivotem w (1).

Złożoność czasowa

W każdym wywołaniu na zbiorze n elementów S , wykonujemy $O(n)$ operacji i wywołujemy select na $\lceil \frac{n}{5} \rceil$ medianach + wywołujemy select na 1 ze zbiorów powstałych po podziale względem pseudomediany p , będącej medianą median piętek. Szacujemy ich moc.



Ile jest co najmniej elementów nie większych od p ?

Wśród $\geq \lceil \frac{1}{2} \lceil \frac{1}{5} n \rceil \rceil - 1$ piątek jest ≥ 3 elementy $\leq p$.
 Do tego w jednej z tych piątek jest p , więc musimy je wykluczyć.

Mamy więc:

$$\geq 3 \left(\lceil \frac{1}{2} \lceil \frac{1}{5} n \rceil \rceil - 1 \right) - 1 \geq \frac{3}{10} n - 4 = \Omega\left(\frac{3}{10} n\right) \text{ elementów nie większych od } p$$

Wniosek p dzieli zbiór na dwie części, najdłuższy ma wielkość $\leq \frac{7}{10} n$, krótszy $\frac{3}{10} n$ czyli mamy:

$$T(n) = T\left(\frac{7}{10} n\right) + T\left(\frac{3}{10} n\right) + O(n)$$

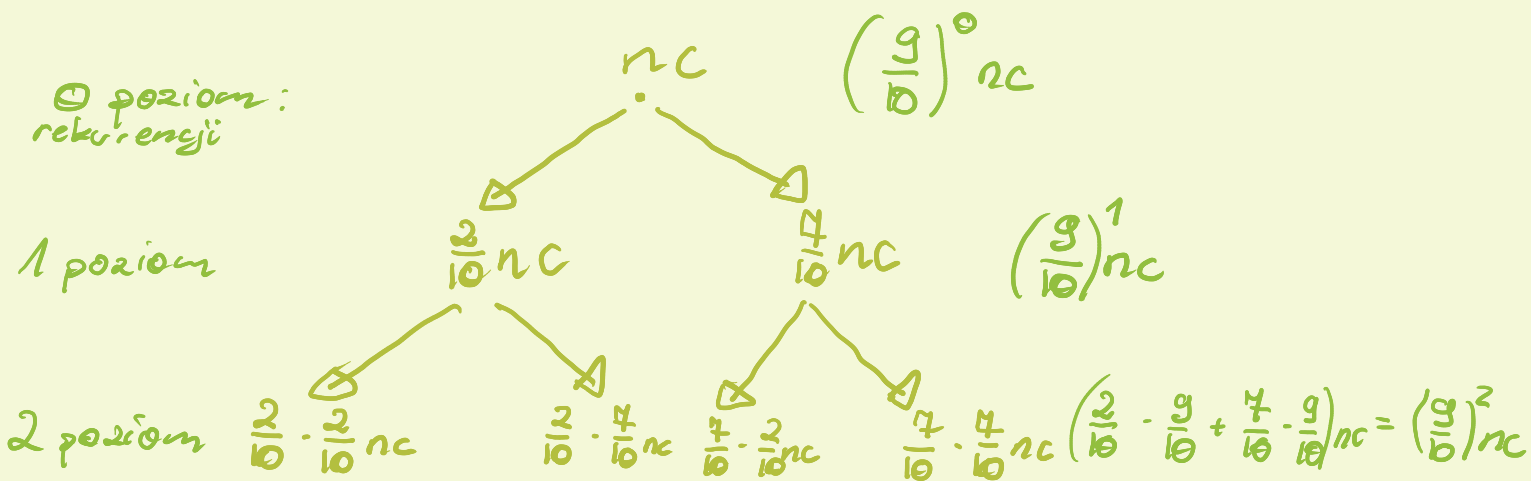
$\uparrow$ szukanie pseudomediany $\uparrow$ największy podzbiór

Jak to policzyć? Niech c oznacza stałą $\in \Theta(1)$

Θ poziom:
rekurencji

1 poziom

2 poziom



Wysuwamy hipotezę, że na i -tym poziomie wykonujemy

$\left(\frac{g}{b}\right)^i n c$ operacji. Dla $i=0$ widzimy na rysunku.

Dalej zauważamy, że w każdym wywołaniu, suma mocy zwońców, dla których mamy wywołanie potomne wynosi $\leq \frac{g}{b} n$ (tutego - przydałoby się lepsze uzasadnienie).

Niech k oznacza liczbę wywołań, stąd:

$$TC = n \cdot \sum_{i=0}^k \left(\frac{g}{b}\right)^i \leq n \cdot \sum_{i=0}^{\infty} \left(\frac{g}{b}\right)^i = n \cdot \frac{1}{1 - \frac{g}{b}} = 10nc = \Theta(n).$$

TW. Niech:

$$T(n) = \begin{cases} \Theta(1) & \text{gdy } n = \Theta \\ \sum_{i=1}^k T\left(\frac{n_i}{m_i} n\right) + \Theta(n) \end{cases}$$

Oraz niech $\sum_{i=1}^k \frac{n_i}{m_i} < 1$. Wtedy $T(n) = \Theta(n)$

d-d: To tylko moja propozycja zadania dla czytelnika.

Nie było tego na wykładzie, ale może to być fajne wyzwanie.

Dlaczego nie trójki?

Elementów $\in \Phi$ jest co najmniej $2(\lceil \frac{1}{2} \lceil \frac{n}{3} \rceil \rceil - 1) - 1 \geq \frac{2}{3}n - 3$

Możemy mieć więc sytuację w którym zrobimy wywołanie $T(\frac{n}{3})$ - medianę median trójek oraz

$T(\frac{2n}{3})$ - większy zbiór - to daje:

$$T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + O(n) = O(n \log n) :-/$$

Siódemki będą (asymptotycznie) tak dobre jak piątki, rozwiązanie ich pozostawiam czytelnikowi.

Cwiczenie - wyznacza S (zbierając mediany od lewej) i parnej i parnej 2 rodzaje pamięci swobodnego dostępu :-)

TSXUY AERSV AAAAA CKNMØ 1YWZŻ
FAVTS AERSV CKXVØ ACPDS BĆDÓZ
AATRU AAAAA M

Oczywiście porządek leksykograficzny :-)

TODO: Lazy sort i randomizedone - one mi nie podchodzą :-/.